Lean-AHR v2.0 Arithmetic Primitives Milestone Report

Verified Proof Search, Arithmetic Bridge, and MVP Interaction Upgrade

Benjamin A. Aenlle Falkenburg

2026-05-31 Europe/Dublin

Abstract

This report documents the Lean-AHR v2.0 arithmetic-primitives milestone completed on 2026-05-31. The run `lean_ahr_v2_0_arith_primitives_repeats27` evaluated 810 Lean-verified proof-search records across top-1, top-2, and top-4 search budgets. It reached 100% proof completion at every tested top-k level, including the new v2.0 families for natural-number inequalities, divisibility, parity encoded as divisibility by 2, and existential witness construction. The run produced a diagnostic atlas, milestone package, JSONL proof records, checkmark logs, and an updated MVP command-line interface that can answer new arithmetic prompts and emit Lean-checked proof files. This is a narrow but meaningful proof-search landmark: AHR is no longer only handling propositional micro-proofs and addition rewrites; it now has a verified bridge into small arithmetic reasoning patterns.

Contents

1 Executive Summary **3**

2 What Lean-AHR Is Doing **3**

3 Mathematical Scope of v2.0 **4**
- 3.1 Inequality . . . 4
- 3.2 Divisibility . . . 4
- 3.3 Parity as Divisibility . . . 4
- 3.4 Existential Witnesses . . . 5

4 Engineering Changes **5**
- 4.1 New v2.0 Runner . . . 5
- 4.2 Detached Execution and Evidence Trail . . . 5
- 4.3 MVP CLI Upgrade . . . 6

5 Experiment Results **6**
- 5.1 Top-k Completion . . . 6
- 5.2 Acceptance Gates . . . 6
- 5.3 Family-Level Coverage . . . 7

6 MVP Smoke Results **7**

7 Artifact Inventory **8**

8 Interpretation **9**

9 Current Limitations **9**

10 Next Research Step: v2.1 **9**
10.1 Recommended v2.1 Families 9
10.2 v2.1 Acceptance 10

11 Path Toward Mathlib and Harder Proofs **10**

12 Conclusion **11**

A Handoff Commands **11**
A.1 Check Remote Status 11
A.2 Run MVP Examples 11
A.3 Archive Location 11
A.4 GitHub Push Already Completed 11

1 Executive Summary

Lean-AHR v2.0 is the first milestone where the proof-search system consistently solved a mixed suite containing both previous propositional/rewriting tasks and new arithmetic primitives. The important result is not merely that the examples compiled in Lean. The important result is that the AHR-guided search loop selected tactic sequences, validated local proof-state transitions with Lean, produced final theorem files, and completed the entire evaluation without failures.

Run name	`lean_ahr_v2_0_arith_primitives_repeats27`
Remote host at completion	`benjamin@35.239.179.142`
Project path	`/home/benjamin/03_ahr_lean_foundation_plan_20260508`
Records	810
Top-k budgets	1, 2, 4
Completion	100% at top-1, top-2, and top-4
Failure count	0
Clinical use flag	`forbidden`
GitHub archive commit	`f0ed3c1`

The immediate interpretation is:

- AHR's Lean proof-search layer is stable on this finite arithmetic/propositional benchmark.
- The top-1 policy is strong on v2.0: it solved all 270 top-1 records.
- The wider search budgets did not expose hidden brittleness: top-2 and top-4 also reached 100%.
- The MVP interface is now able to turn a small set of controlled English prompts into Lean theorem templates and verified proofs.
- The next scientific step is not a bigger demo claim; it is less-templated generalization in v2.1.

2 What Lean-AHR Is Doing

Lean-AHR is being built as a verified reasoning layer around Lean. The working model is:

1. A theorem statement defines a formal target.
2. The current proof state is treated as a finite decision context.
3. Candidate tactics are generated from the local state, theorem family, hypotheses, target shape, and known proof patterns.
4. AHR scores candidate actions using a learned branch-cost/ranking model plus hand-coded safety priors.
5. Lean checks each local step. Invalid branches are rejected by the verifier rather than trusted.
6. If a complete proof is found, a final Lean file is written and compiled without `sorry`.

This architecture deliberately avoids treating the neural model as an oracle. The neural component is used to rank finite choices. Lean is the final source of truth. In other words, AHR proposes; Lean disposes.

3 Mathematical Scope of v2.0

Before v2.0, the strongest Lean-AHR results were primarily in propositional logic, local lemma construction, cases, contradiction, and small natural-number addition rewrites such as associativity and commutativity. v2.0 extends the mathematical surface area to the following natural-number primitives.

3.1 Inequality

The v2.0 inequality family included reflexivity, zero lower bounds, successor bounds, additive bounds, and transitivity:

$$\begin{aligned} n &\leq n, \\ 0 &\leq n, \\ n &< n.\mathrm{succ}, \\ n &\leq n + m, \\ n &\leq m + n, \\ n \leq m \wedge m \leq k &\Rightarrow n \leq k. \end{aligned}$$

The key Lean tactics included direct use of `Nat.le_refl`, `Nat.le_add_right`, `Nat.le_add_left`, and `Nat.le_trans`. This is modest arithmetic, but it matters because it introduces order reasoning and hypothesis chaining beyond equality rewriting.

3.2 Divisibility

The divisibility family introduced goals of the form:

$$\begin{aligned} n \mid n, \\ n \mid m \wedge n \mid k &\Rightarrow n \mid m + k, \\ n \mid m \wedge m \mid k &\Rightarrow n \mid k. \end{aligned}$$

The proof search used Lean primitives such as `Nat.dvd_refl`, `Nat.dvd_add`, and `Nat.dvd_trans`. This extends AHR into relational arithmetic where proof success depends on selecting the right hypothesis pair, not only rewriting a target.

3.3 Parity as Divisibility

Lean core did not expose a convenient `Even`/`Odd` layer in the current no-mathlib setup, so v2.0 encoded parity through divisibility by 2:

$$2 \mid n \Rightarrow 2 \mid n + n.$$

This was solved using `Nat.dvd_add h h`. It is a useful engineering compromise: it keeps the benchmark inside core Lean while still representing a common arithmetic concept.

3.4 Existential Witnesses

The existential family introduced witness construction:

$$\exists k : \mathbb{N},\ k = n.$$

The proof term is short but structurally important:

```
exact Exists.intro n rfl
```

This matters because future theorem proving cannot only discharge universally quantified equations. It must also construct objects. v2.0 adds the first reliable witness-construction path.

4 Engineering Changes

4.1 New v2.0 Runner

The main experiment script was:

```
tools/run_lean_ahr_v2_0_arith_primitives.py
```

The detached launcher was:

```
tools/run_lean_ahr_v2_0_arith_primitives_detached.sh
```

The run was explicitly resumable. This mattered operationally because the VM/IP changed during the job. The run had already written 607 row-level records out of 810. After reconnecting to `35.239.179.142`, the runner resumed from the existing JSONL file rather than restarting the whole experiment. It then completed the remaining top-4 records and wrote the final report.

4.2 Detached Execution and Evidence Trail

The execution pattern produced:

- a PID file,
- detached stdout/stderr log,
- GPU telemetry log,
- 15-minute checkmark snapshots,
- row-level JSONL proof records,
- progress JSON,
- final report JSON,
- diagnostic atlas,
- milestone package with SHA-256 manifest.

This is important for research discipline. A successful proof-search run is not just a console message; it has replayable artifacts, row-level records, and a packaged audit trail.

4.3 MVP CLI Upgrade

The interactive MVP CLI was upgraded:

```
tools/lean_ahr_mvp_cli.py
```

It now supports prompt templates and curated examples for:

- addition associativity and commutativity,
- addition reordering,
- implication chains,
- conjunction projection,
- contradiction,
- disjunction/case-style bridges,
- natural-number inequalities,
- natural-number divisibility,
- parity as divisibility by 2,
- existential witness construction.

The smoke suite:

```
tools/run_lean_ahr_mvp_smoke_suite.sh
```

was expanded to include v2.0 arithmetic prompts and solved all six tested examples.

5 Experiment Results

5.1 Top-k Completion

Top-k	Count	Solved	Completion	Mean Lean calls	Mean elapsed s
1	270	270	1.000	16.20	6.46
2	270	270	1.000	35.93	13.68
4	270	270	1.000	64.53	25.32

The result is unusually clean: all tested search budgets passed. The top-1 score is especially useful because it means the ranker and tactic priors are now choosing the correct first branch across the v2.0 suite, not merely recovering through wider search.

5.2 Acceptance Gates

Gate	Result
top-1 completion ≥ 0.90	pass
top-2 completion ≥ 0.90	pass
top-4 completion ≥ 0.95	pass
zero final Lean failures	pass
clinical use marked forbidden	pass

5.3 Family-Level Coverage

The v2.0 run intentionally retained prior proof families while adding arithmetic primitives. This tests regression as well as new capability.

Family	Count	Solved	Completion	Mean Lean calls
have_construct	27	27	1.000	86.00
have_contradiction	27	27	1.000	22.67
have_imp_chain	27	27	1.000	48.67
have_or_elim	27	27	1.000	46.67
have_projection	27	27	1.000	36.00
v16_deep_chain_conj	27	27	1.000	139.00
v16_nested_or_bridge	27	27	1.000	56.33
v16_or_to_local_conj	27	27	1.000	72.67
v16_project_contradiction	27	27	1.000	57.67
v17_contra_with_distractor	27	27	1.000	83.67
v17_deep_reordered_bridge	27	27	1.000	84.33
v17_distractor_projection	27	27	1.000	47.67
v17_or_with_distractor	27	27	1.000	46.67
v17_reordered_chain	27	27	1.000	52.67
v18_nat_assoc	27	27	1.000	9.33
v18_nat_comm	27	27	1.000	11.67
v18_nat_reorder	27	27	1.000	58.00
v18_nat_rewrite	54	54	1.000	10.17
v20_nat_divisibility	81	81	1.000	23.89
v20_nat_exists	27	27	1.000	8.33
v20_nat_inequality	135	135	1.000	10.40
v20_nat_inequality_trans	27	27	1.000	26.67
v20_nat_parity_div2	27	27	1.000	28.67

6 MVP Smoke Results

After the proof-search run passed, the MVP CLI was updated and tested. The smoke suite solved six out of six examples.

Prompt or task	Proof sketch	Lean calls	Solved
prove addition is associative	`rw [Nat.add_assoc]`	4	yes
prove a natural number is less equal itself	`exact Nat.le_refl n`	4	yes
prove divisibility addition	`intro h0; intro h1; exact Nat.dvd_add h0 h1`	11	yes
prove parity of a double	`intro h0; exact Nat.dvd_add h0 h0`	8	yes
prove existential witness identity	`exact Exists.intro n rfl`	4	yes
custom projection theorem	`intro h0; intro h1; exact h0`	31	yes

The MVP is still a controlled prototype, not an open-ended theorem prover. However, it is now interactive enough to serve as a real demonstration: a user can give a supported prompt or theorem statement and receive a Lean-checked proof file.

7 Artifact Inventory

The milestone package is archived locally and pushed to GitHub.

```
Local archive:
/Users/benjaminfalkenburg/Documents/primumai/github_exports/
  ahr_verified_reasoning_20260509/
    remote_tools/remote_lean_35_239_179_142_20260531/
      v2_0_arith_primitives/
        lean_ahr_v2_0_arith_primitives_repeats27_MILESTONE_PACKAGE_20260531T221022Z/

GitHub:
https://github.com/BoredDuffa/ahr-verified-reasoning-20260509

Commit:
f0ed3c1 Archive Lean AHR v2.0 arithmetic milestone
```

The milestone package contains:

- final report JSON,
- progress report JSON,
- 810-record JSONL proof trace dataset,
- detached logs,
- GPU telemetry logs,
- pre-resume logs,
- checkmark snapshots,
- diagnostic atlas JSON and Markdown,
- updated MVP CLI,
- MVP smoke suite and smoke results,
- SHA-256 manifest.

8 Interpretation

This milestone means AHR has crossed an important narrow threshold. It can now solve and verify a controlled set of Lean theorem-proving tasks that require:

- proposition introduction,
- projection from conjunctions,
- contradiction handling,
- disjunction/case-style bridges,
- local lemma construction,
- addition rewriting,
- inequality facts,
- divisibility facts,
- witness construction for existential goals.

The system is not yet proving advanced number theory. It is not yet proving irrationality of $\sqrt{2}$. But the engineering direction is now clearer: build outward from verified local theorem families, keep Lean as the final verifier, and only claim capability when a held-out proof-search run and an interactive MVP smoke both pass.

9 Current Limitations

- The theorem templates are still narrow. v2.0 success does not prove broad theorem-proving generality.
- The prompt translator is a rule-based template router, not a trained language model.
- The current benchmark mostly uses Lean core. Mathlib-backed theorem proving is a separate next step.
- The system is CPU/Lean-verifier bound during proof search; GPU utilization is naturally low for these runs.
- Search completeness is bounded by max depth, max nodes, tactic candidates, and timeout.
- The benchmark contains validation/test-style splits, but v2.1 needs more aggressive de-templating to test real generalization.

10 Next Research Step: v2.1

The correct next step is Lean-AHR v2.1: less-templated arithmetic generalization.

10.1 Recommended v2.1 Families

1. Reordered inequality transitivity:

$$m \leq k \to n \leq m \to n \leq k.$$

2. Divisibility chains with distractor hypotheses:

$$n \mid m \to x \mid y \to m \mid k \to n \mid k.$$

3. Existential witnesses with nontrivial witness terms:

$$\exists k : \mathbb{N},\ k = n + 0.$$

4. Mixed propositional and arithmetic gates:

$$P \to (P \to n \leq m) \to n \leq m.$$

5. Parity/divisibility goals with irrelevant assumptions.

10.2 v2.1 Acceptance

The proposed gates should be:

- top-1 completion at least 90%,
- top-2 completion at least 95%,
- top-4 completion at least 98%,
- zero Lean expectation mismatches,
- all final proofs compile without `sorry`,
- a failure atlas for every unsolved theorem.

11 Path Toward Mathlib and Harder Proofs

The route from v2.0 to proofs such as irrationality of $\sqrt{2}$ should be staged.

1. Finish v2.1 less-templated arithmetic.
2. Add a Mathlib environment smoke test and import discipline.
3. Build a small Mathlib theorem selection set: divisibility lemmas, parity lemmas, prime facts, square facts.
4. Create a retrieval layer for named lemmas.
5. Evaluate local proof search where AHR chooses among lemma applications.
6. Only then attempt multi-lemma proofs such as irrationality of $\sqrt{2}$.

The mathematical challenge in $\sqrt{2}$ is not simply arithmetic. It requires a chain of definitions and lemmas about irrationality, divisibility, parity, squares, primes, and contradiction. Lean can verify such proofs, but AHR needs a controlled way to discover the required lemmas and route through them.

12 Conclusion

Lean-AHR v2.0 is a clean milestone. The run completed 810 Lean-verified proof-search records with 100% completion at top-1, top-2, and top-4. It introduced working arithmetic primitives, preserved earlier propositional capability, upgraded the MVP interaction path, and produced a packaged, checksummed artifact set pushed to GitHub.

The next milestone should deliberately stress generalization. v2.1 should keep the same verifier discipline but make the arithmetic proofs less templated, more reordered, and more distractor-heavy. If that passes, the project will be ready for its first serious Mathlib bridge.

A Handoff Commands

A.1 Check Remote Status

```
ssh benjamin@35.239.179.142
cd /home/benjamin/03_ahr_lean_foundation_plan_20260508
ls -lh reports/lean_ahr_v2_0_arith_primitives_repeats27_report.json
tail -n 40 reports/lean_ahr_v2_0_arith_primitives_repeats27.detached.log
```

A.2 Run MVP Examples

```
cd /home/benjamin/03_ahr_lean_foundation_plan_20260508
/opt/conda/bin/python tools/lean_ahr_mvp_cli.py --list-examples
/opt/conda/bin/python tools/lean_ahr_mvp_cli.py \
  --prompt "prove divisibility addition" \
  --name demo_dvd_add \
  --run-name lean_ahr_mvp_demo_dvd_add \
  --max-k 1 \
  --cpu
```

A.3 Archive Location

```
/Users/benjaminfalkenburg/Documents/primumai/github_exports/
  ahr_verified_reasoning_20260509/
    remote_tools/remote_lean_35_239_179_142_20260531/
```

A.4 GitHub Push Already Completed

```
cd /Users/benjaminfalkenburg/Documents/primumai/github_exports/
    ahr_verified_reasoning_20260509
git log --oneline -1
# f0ed3c1 Archive Lean AHR v2.0 arithmetic milestone
```